

Block 3b: Praxis II — Reviews, Bugfixing, Abschluss

Workshop: Agentic Engineering

Thomas Hein — Datacidars

Code Reviews mit dem Agenten

Der Agent als zusätzlicher Reviewer:

- Prüft: Konventionen, Security, Best Practices, Architektur
- Findet: Was Menschen übersehen (Security-Lücken, vergessene Edge Cases)
- Liefert: Konsistente Reviews nach definierten Regeln

Review-Kommentare schrittweise abarbeiten

1. Agent holt die **Liste** aller Review-Kommentare
2. Pro Kommentar:
 - Agent **schlaegt** Lösungsweg vor
 - Entwickler **bestaetigt** oder **aendert**
 - Agent **implementiert** + **testet**
 - **Commit**
3. Agent verfasst **Antwort** auf den Kommentar

Nicht alle auf einmal — Schritt fuer Schritt!

Quellen & Weiterlesen — Code Reviews mit Agenten

- [CodeRabbit — AI Code Reviews](#) — Automatische PR-Reviews auf GitHub & GitLab, Multi-Layer-Analyse, agentic @coderabbitai-Interaktion
- [Claude Code — Code Review \(Docs\)](#) — Multi-Agent-Review mit Confidence Scoring, Inline-Kommentare auf GitHub PRs
- [GitHub Copilot Code Review \(Docs\)](#) — Agentic Review mit vollem Repository-Kontext, CLI-Integration via `gh pr`
- [Request Copilot review from GitHub CLI](#) — Copilot-Review direkt aus dem Terminal starten
- [State of AI Code Review Tools 2025](#) — Vergleich der wichtigsten Tools

Bugfixing: Hypothesenbasiert

70% Analyse — 30% Fix

1. **Kontext sammeln:** Logs, Fehlermeldung, betroffene Module
2. **Hypothese bilden:** "Der Bug entsteht weil..."
3. **Failing Test:** Test reproduziert den Bug → ROT
4. **Minimaler Fix:** Kleinstmögliche Aenderung → GRUEN
5. **Absichern:** Regression-Tests + Clean-Code-Check

Bugfixing: Wann Agent, wann nicht?

Szenario	Agent-Eignung
Klarer Stacktrace, reproduzierbar	✅ Gut
Regressions-Bug	✅ Gut
Komplexe Geschäftslogik	⚠️ Bedingt
Intermittierend / Race Condition	❌ Schwierig
Performance-Bug	❌ Schwierig

Quellen & Weiterlesen — Bugfixing mit Agenten

- [Test-driven development \(TDD\) with GitHub Copilot](#) — TDD-Workflow mit Copilot, schrittweise Anleitung fuer Einsteiger
- [How to Use TDD for better AI coding outputs](#) — TDD als Qualitaetssicherung fuer KI-generierten Code
- [How I Validate Quality When AI Agents Write My Code](#) — Praxisbericht: Qualitaetssicherung und Bugfixing mit KI-Agenten

Sandboxing: Die richtigen Stopps setzen

Das Problem

Zu viele Stopps → "Accept All" → kein Schutz

Zu wenige Stopps → Agent entscheidet allein → Kontrollverlust

Ziel: Nur bei wichtigen Dingen stoppen.

Welche Aktionen brauchen welchen Schutz?

	Aktion	Warum
✅ Auto	Dateien lesen/editieren, lokale Tests	Reversibel via Git
✅ Auto	Git add/commit (lokal)	Reversibel
⚠️ Fragen	Architektur-Entscheidung, neue Dependency	Nicht-trivial reversibel
⚠️ Fragen	DB-Schema, oeffentliche API	Breaking Change
🚫 Block	Netzwerk, Datenbank, SSH, Secrets	Security-kritisch
🚫 Block	Git push, sudo	Oeffentlich / Systemrisiko

Sandbox konfigurieren — tooluebergreifend

Schicht 1: AGENTS.md (alle Tools)

```
## Grenzen & No-Gos  
Always: Dateien lesen, editieren, Tests ausfuehren  
Ask first: Neue Dependencies, DB-Schema, API-Aenderungen  
Never: Netzwerk, Secrets, Push ohne Review
```

Schicht 2: Tool-spezifische Config

- Claude Code: `.claude/settings.json` (im Repo versionierbar)
- Copilot CLI: Workspace Trust + Copilot Instructions

Schicht 3: Team-Profil bereitstellen

- Config im Repo → `git clone` = fertig konfiguriert

Quellen & Weiterlesen — Sandboxing und Berechtigungssteuerung

- [Claude Code: Configure Permissions \(Docs\)](#) — allowedTools/blockedTools, deny-Regeln, Wildcard-Syntax fuer `.claude/settings.json`
- [Claude Code Security Best Practices — Backslash](#) — Praxisleitfaden: Filesystem-Restrictions, Secrets-Management, Sandbox-Konfiguration
- [VS Code Copilot Security \(Docs\)](#) — Workspace Trust, Agent Sandboxing, MCP-Server-Sicherheit
- [Safeguarding VS Code against prompt injections — GitHub Blog](#) — Angriffsvektoren auf Agent Mode und Gegenmassnahmen
- [Knostic — AI Coding Agent Governance Policies That Work](#) — Team-Governance: Policies definieren, durchsetzen, reviewen

Euer Werkzeugkasten: Was eignet sich?

Eignung	Aufgabentyp
✅ Gut	CRUD, UI, Boilerplate, Refactoring, Demonstratoren, Library Updates
⚠️ Bedingt	Bugfixing, Integration, Legacy-Code
❌ Schwierig	Komplexe Algorithmik, Performance, tiefe fachliche Logik

AI Code Archaeology

Bestehende Codebasen verstehen und unter Kontrolle bringen

- Agent liest und erklart bestehenden Code
- Erzeugt Dokumentation fuer undokumentierte Module
- Hilft beim Onboarding in neue Projekte
- Identifiziert Patterns, Antipatterns, Abhaengigkeiten

Besonders wertvoll fuer unsere 20-Jahre-Plattform!

Das Best-Practice-Repo

```
vortrag-agentic-ai/  
  best-practices/  
    agents-md/      ← AGENTS.md Vorlagen  
    skills/         ← Evaluierte Workflows  
    plugins/        ← Plugin-Empfehlungen  
    kontext-management/ ← Kontext-Schichten  
    dokumentation/  ← Doku-Vorlagen  
    antipatterns/    ← Was NICHT tun  
    use-cases/       ← Übungsaufgaben
```

Lebendes Dokument — das Team pflegt es weiter!

Kosten im Blick

- **Copilot Business:** 300 Premium-Requests/Monat inklusive
- **Premium-Modelle** (Claude, o1) verbrauchen mehr Requests
- **Einsparen:** Richtiges Modell waehlen, guten Kontext liefern, Standard-Modelle fuer einfache Aufgaben
- **ROI:** Weniger Nacharbeit durch Methodik > Kosten fuer Premium-Requests

Details: `referenzen/modell-vergleich.md`

Hands-on: Aufgaben fuer die Projektarbeit

1. Einfach (Vibe Coding reicht):

UI-Text aendern, Config anpassen

2. Mittel (Spec-Driven):

Neuen API-Endpoint mit Tests erstellen

3. Komplex (Agentic Engineering):

Neues Modul mit Architektur-Entscheidungen

Use Cases mit Contact-Software-Kontext: `use-cases/`

Quellen & Weiterlesen — Abschluss und Ausblick

- [Addy Osmani: My LLM Coding Workflow Going Into 2026](#) — Praxisworkflow eines Staff-Engineers bei Google: Brainstorming, Spec, Execution, Review
- [Anthropic: Agentic Coding Trends 2026](#) — Marktentwicklung, Team-Strukturen und Produktivitätsdaten aus der Praxis
- [Martin Fowler: Humans and Agents in SE Loops](#) — In-the-loop, on-the-loop, out-of-the-loop: Kontrolle und Delegation im Agentic-Workflow
- [Thoughtworks Tech Radar: Spec-Driven Development](#) — Etablierter Ansatz fuer strukturierte Entwicklung mit klaren Spezifikationen
- [GitHub Copilot Workspace](#) — Spec-to-Code Workflow direkt in GitHub: von Issue bis Pull Request

Der Entwickler ist der Boss. Der Agent ist das Werkzeug.

*"You are orchestrating agents who do and acting as oversight."
— Andrej Karpathy, 2026*